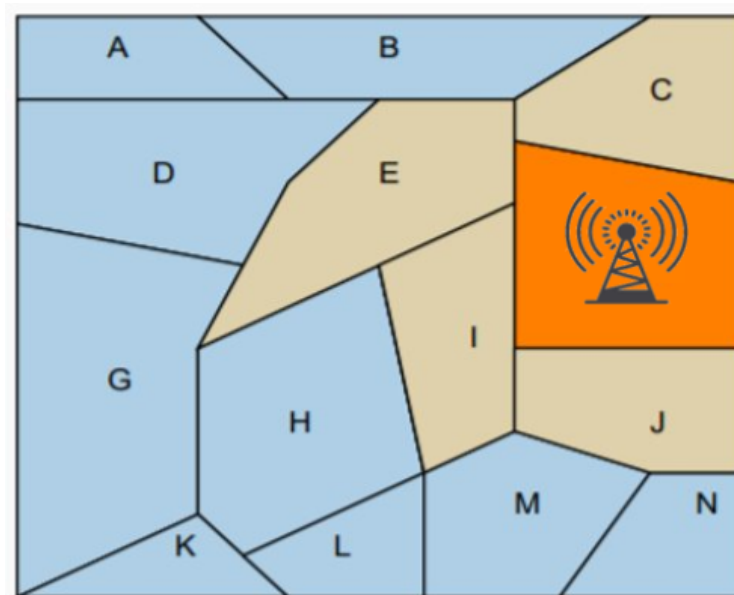


Formalisation & programmation linéaire

Dans les exercices qui suivent nous allons partir d'énoncés déjà simplifiés à partir d'un problème réel. Le but sera de formaliser ces problèmes, puis de les traduire en un programme linéaire. Enfin il faudra généraliser les problèmes proposés.

Exercice 1 : Surveillance à bas coût

Un radar positionné sur un secteur de la carte ci-dessous surveille également les secteurs voisins. Comment placer des radars pour surveiller toute la zone, avec le moins de radar possible ?



Question 1-1

En langage mathématique, formaliser ce problème en répondant aux questions suivantes :

- Quelles sont les données du problème ?
- Quelle est la forme de la sortie attendue, c'est-à-dire, quelles sont les variables auxquelles il faut trouver une valeur pour avoir une sortie potentielle ?
- Quelles sont les contraintes que doit respecter la sortie ?
- Quel est l'objectif à optimiser ?

Question 1-2

Traduire la formalisation en langage mathématique précédente en un programme linéaire.

Question 1-3

1. Généraliser le problème spécifique précédent en un énoncé général (une carte découpée en zones voisines ou non, avec les mêmes propriétés pour un radar).
2. Écrire la formalisation en langage mathématique de cet énoncé général.
3. En vous aidant de la documentation d'OR-tools ci-dessous, écrire un programme linéaire qui résout ce problème généralisé.

```
from ortools.linear_solver import pywraplp

def main():
    # Create the mip solver with the SCIP backend.
    solver = pywraplp.Solver.CreateSolver("SAT")
    if not solver:
        return

    infinity = solver.infinity()
    # x and y are integer non-negative variables.
    x = solver.IntVar(0.0, infinity, "x")
    y = solver.IntVar(0.0, infinity, "y")

    print("Number of variables =", solver.NumVariables())

    # x + 7 * y <= 17.5.
    solver.Add(x + 7 * y <= 17.5)

    # x <= 3.5.
    solver.Add(x <= 3.5)

    print("Number of constraints =", solver.NumConstraints())

    # Maximize x + 10 * y.
    solver.Maximize(x + 10 * y)

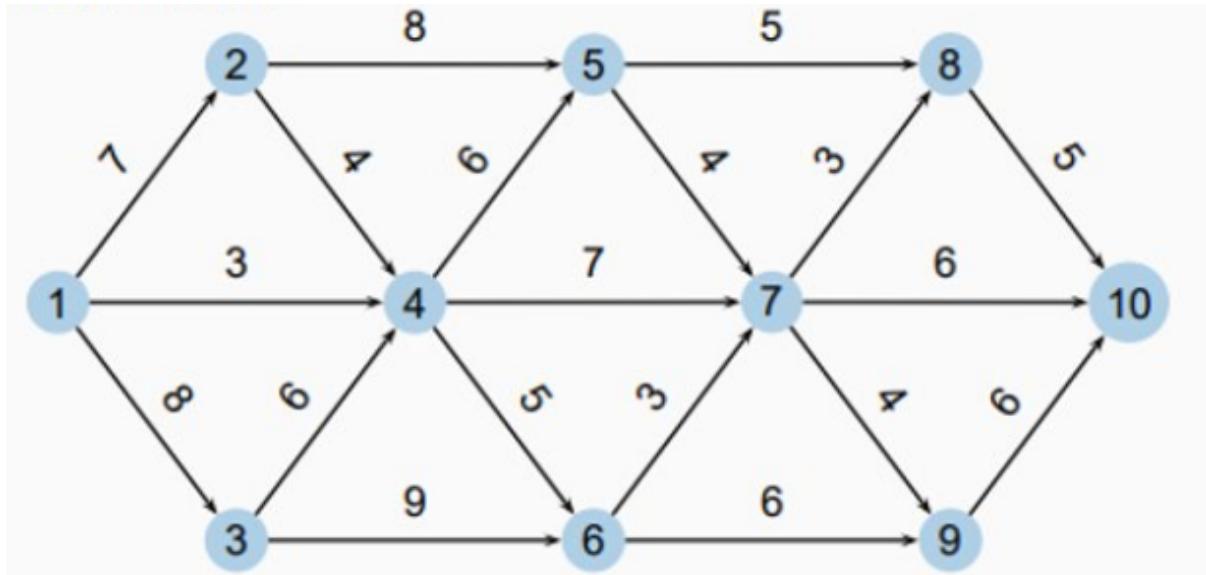
    print(f"Solving with {solver.SolverVersion()}")
    status = solver.Solve()
```

https://developers.google.com/optimization/mip/mip_example

Exercice 2 : Le plus court chemin

Cet exercice suit les mêmes questions que l'exercice précédent. Vous pouvez essayer de le généraliser immédiatement, si vous voulez.

Nous cherchons le plus court chemin entre les nœuds 1 et 10 du graphe ci-dessous.



Questions

1. En langage mathématique, formaliser ce problème en répondant aux questions déjà vues à l'exercice 1.
2. Traduire la formalisation en langage mathématique précédente en un programme linéaire.
3. Généraliser le problème spécifique précédent en un énoncé général (un graphe orienté avec un nœud de départ et un nœud d'arrivée).
4. Écrire la formalisation en langage mathématique de cet énoncé général.
5. En vous aidant de la documentation d'OR-tools ci-dessus, écrire un programme linéaire qui résout ce problème généralisé.

Exercice 3 : le projet ERO1

Vous devrez montrer à votre coach la formalisation que vous faites du problème proposé dans le projet ERO1.

Les étapes de formalisation proposées dans ce TD peuvent vous servir de guide pour cela.